Universidad Nacional de Ingeniería.

Recinto Universitario “Simón Bolívar”.

DIRECCIÓN DEL ÁREA DE CONOCIMIENTO DE LA TECNOLOGÍA DE LA INFORMACIÓN Y LA COMUNICACIÓN.

INGENIERÍA DE SISTEMAS.

Programación II

Tema: Procesamiento paralelo, concurrencia y sincronia.

Integrantes: Abel Melquisedec Bermúdez Díaz

Maikeli Guivelki Rodriguez Centeno

David Alberto Reyes Castillo

Grupo: 2T1-SIS-S.

Docente: Msc Abel Edmundo Marin Reyes.

Managua, Nicaragua 12 de Abril de 2026

Clase Venta

Entidad Principal del Sistema, representa a las ventas.

Propiedades:

Id: Identificador único

Producto: Nombre del producto

Categoria: Tipo de producto

Cantidad: Unidades vendidas

PrecioUnitario: Precio por unidad

Fecha: Fecha de la venta

Propiedad calculada:
Total = Cantidad * PrecioUnitario

```
public class Venta
{
 public int Id { get; set; }
 public string Producto { get; set; }
 public string Categoria { get; set; }
 public int Cantidad { get; set; }
 public decimal PrecioUnitario { get; set;
}public DateTime Fecha { get; set; }
 public decimal Total
 {
 get { return Cantidad * PrecioUnitario; }
 }
}
```

Interfaz IventaRepository

Define el contrato para obtener ventas.

Método:

ObtenerVentasAsync(): retorna una lista de ventas de forma asíncrona.

```
public interface IVentaRepository
{
 Task<List<Venta>>
ObtenerVentasAsync();
}
```

Clase VentaRepository

Se encarga de simular la obtención de datos.

Usa Task.Delay(2000) para simular carga.

Retorna una lista de ventas de ejemplo.

```
public class VentaRepository : IVentaRepository
{
 public async Task<List<Venta>> ObtenerVentasAsync()
 {
 await Task.Delay(2000);
 return new List<Venta>
 {
 new Venta { Id = 1, Producto = "Laptop", Categoria = "Tecnología", Cantidad = 2, PrecioUnitario
= 550m, Fecha = DateTime.Now.AddDays(-1) },
 new Venta { Id = 2, Producto = "Mouse", Categoria = "Tecnología", Cantidad = 5, PrecioUnitario
= 20m, Fecha = DateTime.Now.AddDays(-2) },
 new Venta { Id = 3, Producto = "Teclado", Categoria = "Tecnología", Cantidad = 3, PrecioUnitario
= 35m, Fecha = DateTime.Now.AddDays(-3) },
 new Venta { Id = 4, Producto = "Silla", Categoria = "Oficina", Cantidad = 2, PrecioUnitario =
120m, Fecha = DateTime.Now.AddDays(-2) },
 new Venta { Id = 5, Producto = "Escritorio", Categoria = "Oficina", Cantidad = 1, PrecioUnitario =
250m, Fecha = DateTime.Now.AddDays(-4) },
 new Venta { Id = 6, Producto = "Monitor", Categoria = "Tecnología", Cantidad = 4, PrecioUnitario
= 180m, Fecha = DateTime.Now.AddDays(-1) }
 };
 }
}
```

Clase VentaService

Procesa las ventas y genera un reporte.

Funciones principales:

Total vendido (Sum)

Promedio (Average)

Venta mayor (OrderByDescending)

Agrupación por categoría (GroupBy)

Construcción del reporte con StringBuilder

```
public class VentaService
{
 public async Task<string> ProcesarVentasAsync(List<
 {
 return await Task.Run(() =>
 {
 StringBuilder sb = new StringBuilder();
 decimal totalVendido = ventas.AsParallel().Sum(v =>
 decimal promedio = ventas.AsParallel().Average(v =>
 Venta ventaMayor = ventas.AsParallel()
 .OrderByDescending(v => v.Total)
 .FirstOrDefault();

   var agrupadas = ventas.AsParallel()
 .GroupBy(v => v.Categoria)
 .Select(g => new
 {
 Categoria = g.Key,
 TotalCategoria = g.Sum(x => x.Total),
 CantidadVentas = g.Count()
 })
 .ToList();
 sb.AppendLine("===== REPORTE DE VENTAS =====");
 sb.AppendLine($"Total vendido: {totalVendido:C}");
 sb.AppendLine($"Promedio por venta: {promedio:C}");
 if (ventaMayor != null)
 {
 sb.AppendLine($"Venta mayor: {ventaMayor.Producto} - {ventaMayor.Total:C}");
 }
 sb.AppendLine();
 sb.AppendLine("Ventas por categoría:");
 foreach (var item in agrupadas)
 {
 sb.AppendLine($"- {item.Categoria}: {item.TotalCategoria:C} en {item.CantidadVentas}
ventas");
 }
 return sb.ToString();
 });
 }
}
```

```
public partial class Form1 : Form
{
 private readonly IVentaRepository
_ventaRepository;
private readonly VentaService
_ventaService;
 private List<Venta> _ventas;
 public Form1()
 {
 _ventaRepository = new
VentaRepository();
 _ventaService = new VentaService();
 InitializeComponent();
 }
```

Formulario Form1

Interfaz gráfica del sistema.

Variables:

IventaRepository: obtiene datos

VentaService: procesa datos

List<Venta>: almacena ventas

Botón Cargar

Carga las ventas desde el repositorio.

Muestra estado de carga

Llena el DataGridView

Maneja errores

```
private async void
btnCargar_Click(object sender,
EventArgs e)
{
 try
 {
 lblEstado.Text = "Cargando ventas...";
 progressBar1.Style =
ProgressBarStyle.Marquee;
 btnCargar.Enabled = false;
 btnProcesar.Enabled = false;
 _ventas = await
_ventaRepository.ObtenerVentasAsync();
dgvVentas.DataSource = null;
 dgvVentas.DataSource = _ventas;
 lblEstado.Text = "Ventas cargadas
correctamente.";
 }
 catch (Exception ex)
 {
 MessageBox.Show("Ocurrió un error al
cargar ventas: " + ex.Message);
 }
 finally
 {
 progressBar1.Style =
ProgressBarStyle.Blocks;
 btnCargar.Enabled = true;
 btnProcesar.Enabled = true;
 }
}
```

Botón Procesar

Genera el reporte de ventas.

Valida que existan datos

Procesa información

Muestra el resultado

```
private async void
btnProcesar_Click(object sender,
EventArgs e)
{
 if (_ventas == null || _ventas.Count
== 0)
 {
 MessageBox.Show("Primero debe cargar
las ventas.");
 return;
}
 try
 {
 lblEstado.Text = "Procesando
información...";
 progressBar1.Style =
ProgressBarStyle.Marquee;
 btnProcesar.Enabled = false;
 string reporte = await
_ventaService.ProcesarVentasAsync(_vent
as);
 txtResultado.Text = reporte;
 lblEstado.Text = "Procesamiento
finalizado.";
 }
 catch (Exception ex)
 {
 MessageBox.Show("Ocurrió un error al
procesar las ventas: " + ex.Message);
 }
 finally
 {
 progressBar1.Style =
ProgressBarStyle.Blocks;
 btnProcesar.Enabled = true;
 }
}
```

Botón Limpiar

Limpia la información del sistema.

Borra tabla

Borra resultados

Reinicia estado

```
private void btnLimpiar_Click(object
sender, EventArgs e)
 {
  dgvVentas.DataSource = null;
 txtResultado.Clear();
 lblEstado.Text = "Sin datos";
 _ventas.Clear();
 }
```

Esta clase, denominada VentasRepositoryJson, actúa como una capa de persistencia dentro del espacio de nombres Clase_practica.Repositorios, encargándose de la gestión de archivos de datos en formato JSON. Su función principal es la serialización de una lista de objetos de tipo Venta para almacenarlos físicamente en el disco, utilizando para ello la constante rutarchivo que apunta al archivo "ventas.json". El núcleo de su funcionamiento reside en el método asíncrono ObtenerVentasAsync(List<Venta> listaVenta), el cual emplea la librería Newtonsoft.Json para transformar la lista de objetos en una cadena de texto estructurada con sangría y, posteriormente, utiliza File.WriteAllTextAsync para escribir dicha cadena en el archivo de forma no bloqueante.

Clase_practica.Repositorios.VentasRepositoryJson

```
using Clase_practica.Entidades;
using System;
using System.Collections.Generic;
using System.Text;
using System.IO;
using Newtwnsoft.Json;
namespace Clase_practi.Repositoios
{
  public class VentasRepositoryJson : IVentasrepository
  {
    private readonly string rutarchivo = "ventas.json";


     public async Task ObtenerVentasAsync(List<Venta> listaVenta)
     {
        string json = JsonConvert.SerializeObject(ListaVenta, Formatting.Indented);
        await File.WriteAllTextAsync(rutarchivo, json);
    } }

    public Task<List<ventas>> ObtenerVentasAsync()
    {
      throw new NotImplementedException("No se usa deserialización.");
    }
  }
}
```

Por otro lado, la clase incluye una sobrecarga del método ObtenerVentasAsync() destinada a la lectura y deserialización de datos, aunque actualmente se encuentra deshabilitada mediante una excepción NotImplementedException que indica que no se está utilizando dicho proceso. Cabe destacar que para el correcto funcionamiento de este repositorio es indispensable que el proyecto cuente con las referencias adecuadas a System.IO y el paquete NuGet de Newtonsoft.Json. Además, existen detalles menores por corregir en el código, como la errata en el nombre del espacio de nombres y la capitalización de la entidad Venta en el tipo de retorno del segundo método.